

흐름제어와 컬렉션

학습 체크리스트 (1/2)

☐ Java 조건식의 엄격한 Boolean 평가 원리 및 Truthy/Falsy 에러 이해

☐ switch 표현식의 화살표 연산자(`->`) 활용 및 값 반환 특징 습득

☐ 반복 제어문 내 break와 continue의 흐름 제어 차이 명확히 구분

☐ 일반 for문 선언 시 루프 변수의 명시적 타입 지정 규칙 숙지

☐ 다차원 반복 제어를 위한 중첩 for문의 실행 순서 및 제어 흐름 분석

학습 체크리스트 (2/2)

- ☐ 자바 배열의 두 가지 핵심적 물리적 제약사항(단일 타입, 고정 크기) 이해
- ☐ 배열의 선언, 공간 할당, 리터럴 초기화 및 크기 조회(`.length`) 활용법 습득
- ☐ 향상된 for문(for-each)의 읽기 전용(Read-only) 특성 파악 및 순회 적용
- ☐ 컬렉션 프레임워크와 제네릭(Generic) 타입 제한의 기초 메커니즘 이해
- ☐ ArrayList, HashMap, HashSet 자료구조별 데이터 삽입 및 삭제 메서드 조작법 숙지

흐름제어 개요

- **공통점:** 조건문과 반복문의 기본 골격 및 흐름제어 논리는 JavaScript와 매우 유사함
- **차이점:** 타입 검사가 고도로 엄격한 정적 타입 언어 특성상, 제어 변수의 선언 규격이나 조건 평가 기준에서 강한 문법적 제약이 발생함
- **교육 목표:** 본 단원에서는 JS 경험을 바탕으로, 오직 자바에만 존재하는 고유 제약사항과 모던 자바 문법 트렌드(Switch Expressions 등)에 집중함

if문 조건식 내부의 Boolean 제약

- Truthy/Falsy 불인정: 자바는 JavaScript와 달리 조건문 내부에서 숫자가 `0` 이거나 객체가 `null` 이라고 해서 거짓(`false`)으로 자동 해석하지 않음
- 타입 일치 강제: `if` 문의 소괄호 내부에는 오직 실행 결과가 최종적으로 `boolean` 타입(`true` 또는 `false`)인 연산식만 들어갈 수 있음
- 문법 차단: `if (1)` 또는 `if (obj)` 등은 엄격하게 컴파일 차단되며, 비교 연산자나 논리 연산자를 사용한 명확한 식을 작성해야 함

조건문 내 명확한 Boolean 식 평가

```
1 public static void main(String[] args) {  
2     int count = 0;  
3     // if (count) → 컴파일 에러 유발  
4     if (count == 0) {  
5         System.out.println("값이 0입니다.");  
6     }  
7 }
```

switch문과 Fall-through 현상

- 전통적 switch문: 특정 값의 일치 여부에 따라 분기 처리를 수행하는 다중 조건문
- Fall-through: `case` 블록마다 제어 흐름을 끊어주는 `break` 명령어를 명시적으로 달아주지 않으면, 하단에 기재된 다른 `case` 블록의 코드까지 멈추지 않고 연이어 밀고 실행되는 현상
- 오동작 방지: 의도한 경우가 아니라면 반드시 각 case의 동작 끝에 `break;`를 삽입하여 분기 흐름을 제어해야 함

switch 표현식 (Switch Expressions)

- **문법 혁신:** 전통적인 switch문의 불편을 개선하여 화살표 연산자(`->`)를 도입한 새로운 switch 블록 형식
- **자동 break 동작:** 화살표 연산자(`->`)를 사용하면 동작을 마친 뒤 하단 case로 흘러내리는 Fall-through 현상이 발생하지 않고 즉시 블록 탈출
- **식(Expression) 평가:** 블록 전체가 최종적으로 **하나의 값**으로 계산되며, 이를 변수에 바로 대입하거나 반환값으로 재 활용 가능
- **침표 다중 나열:** 여러 case 조건을 침표(`,`)로 구분하여 나란히 나열할 수 있어 소스 코드 가독성이 우수함

switch 표현식을 이용한 성적 평가

```
1 public static void main(String[] args) {  
2     String grade = "A";  
3     String result = switch (grade) {  
4         case "A", "B" → "우수";  
5         case "C"      → "보통";  
6         default       → "재수강";  
7     };  
8     System.out.println(result);  
9 }
```

while문과 break / continue 제어자

- **while문**: 조건식의 결과가 참(`true`)인 동안 중괄호 블록 내의 실행문을 반복해서 구동
- **break 제어자**: 반복 루프 내에서 특정 조건을 만족할 때 루프 자체를 즉각 파괴하고 반복 영역을 완전히 이탈
- **continue 제어자**: 루프 블록 내의 남은 실행 코드를 생략하고, 다음 회차의 반복 수행 여부를 판별하기 위해 즉시 조건 검사식으로 점프

while 루프 내 break 및 continue 제어

```
1 public static void main(String[] args) {  
2     int i = 0;  
3     while (i < 5) {  
4         i++;  
5         if (i == 3) continue;  
6         System.out.println(i);  
7         if (i == 4) break;  
8     }  
9 }
```

for문과 루프 변수 타입 명시

- **for문**: 횟수를 카운팅하거나 일정 규칙에 맞추어 인덱스를 증감시키며 반복 영역을 통제하는 구문
- **루프 제어 변수**: JavaScript의 `let` 선언과 달리, 자바의 for문 초기화 식에서는 루프 인덱스 변수의 타입을 반드시 명시적으로 표기해야 함 (예: `int i = 0`)
- **변수 유효 영역**: for문 초기화 식에서 선언된 변수는 오직 해당 for문 블록 내부에서만 존재하며, 루프 종료 시 메모리에서 완전히 소각됨

중첩 for문의 정의와 다중 루프

- 중첩 루프: `for` 문 내부에 또 다른 `for` 문을 설계하여 구동시키는 다차원 반복 제어 기법
- 동작 흐름: 바깥쪽 `for` 문이 1회 전개될 때, 안쪽에 설계된 `for` 문은 지정된 전체 횟수만큼 온전히 회전을 마쳐야 바깥쪽 루프의 다음 회차로 진행함
- 주의점: 루프 차원이 늘어날 때마다 계산 복잡도가 급격히 상승하므로, 성능 병목 현상이 발생하지 않는지 고려하여 작성해야 함

구구단 2단을 이용한 중첩 for문 제어

```
1  public static void main(String[] args) {  
2      for (int dan = 2; dan ≤ 2; dan++) {  
3          for (int i = 1; i ≤ 3; i++) {  
4              System.out.println(dan + "*" + i + "=" + (dan * i));  
5          }  
6      }  
7  }
```

자바 배열의 두 가지 물리적 제약

- **JS 배열과의 차이:** 자바스크립트 배열은 타입이 유연하고 크기가 동적으로 늘어나지만, 자바 배열은 기계적인 한계를 엄격히 유지함
- **1. 단일 타입 보존:** 배열을 선언할 때 지정한 고유의 데이터 타입 요소만 저장할 수 있으며, 이종 타입 혼합 보관은 절대 불가
- **2. 고정 크기 제한:** 배열은 런타임에 인스턴스를 생성하는 순간 물리적으로 방의 크기(길이)가 확정되며, 이후 요소를 추가 또는 삭제하더라도 크기가 유동적으로 늘어나거나 줄어들지 않음

배열의 선언, 할당, 리터럴 초기화

- 배열 선언: 보관하려는 타입명 뒤에 대괄호 `[]` 를 선언하여 배열 참조 변수 정의 (예: `int[] numbers;`)
- 공간 할당: `new` 키워드와 함께 보관할 칸수를 명시하여 메모리 할당 (예: `numbers = new int[5];`)
- 리터럴 초기화: 할당과 동시에 구체적인 데이터 목록을 중괄호 `{}` 안에 명시하여 한 번에 채워 넣음 (예: `int[] numbers = {10, 20, 30};`)

인덱스 범위 초과 및 .length 속성

- 인덱스 접근: 배열의 첫 번째 칸은 항상 `0` 번 인덱스부터 시작하며 대괄호를 통해 접근 (예: `numbers[0]`)
- 인덱스 범위 예외: 할당된 크기를 초과하여 존재하지 않는 인덱스 공간에 접근할 경우, 런타임에 심각한 `ArrayIndexOutOfBoundsException` 예외가 발생
- 배열의 크기 조회: 배열 참조 변수 뒤에 `.length` 속성을 참조하면, 해당 배열이 가진 물리적 고정 크기값을 바로 확인 가능 (예: `numbers.length`)

1차원 정수 배열 선언 및 인덱스 제어

```
1 public static void main(String[] args) {  
2     int[] numbers = {10, 20, 30};  
3     numbers[0] = 100;  
4     for (int i = 0; i < numbers.length; i++) {  
5         System.out.println(numbers[i]);  
6     }  
7 }
```

향상된 for문 (for-each)의 특성

- **for-each 구문:** 배열이나 컬렉션의 요소를 처음부터 끝까지 인덱스 변수 제어 없이 순차적으로 꺼내어 주는 편리한 제어문
- **가독성 향상:** 복잡한 인덱스 루프 변수 계산이나 배열 길이 조회가 불필요해져 단순 조회 목적의 루프 코드가 간결해짐
- **읽기 전용 (Read-only):** 루프 내에서 꺼내어 선언된 변수는 배열 속 실제 요소의 복사본이므로, 해당 변수 값을 변경해도 원본 배열 내부 요소의 값은 전혀 바뀌지 않음

for-each 루프를 활용한 배열 순회

```
1 public static void main(String[] args) {  
2     int[] scores = {90, 80, 100};  
3     for (int score : scores) {  
4         System.out.println(score);  
5     }  
6 }
```

컬렉션 프레임워크 개요

- 도입 목적: 크기가 고정된 일반 배열의 한계를 극복하고, 동적으로 변하는 객체 무리를 유연하게 관리하기 위한 자바 표준 프레임워크
- 주요 인터페이스 계열:
 - **List**: 순서가 유지되고 중복 저장을 허용하는 선형 자료구조
 - **Map**: 키(Key)와 값(Value)의 쌍으로 묶어 데이터를 매핑하는 자료구조
 - **Set**: 순서를 보장하지 않으며, 중복 요소를 절대 보관할 수 없는 고유 집합

제네릭 (Generic) 타입 안전 장치

- **정의:** 컬렉션 클래스나 변수를 선언할 때, 꺾쇠괄호 `< >` 내부에 명시하여 다를 수 있는 데이터 타입을 컴파일 시점에 한정시키는 문법
- **도입 취지:** 컴파일러가 컬렉션 내부로 들어가는 데이터의 타입을 엄격히 통제하여, 이종의 잘못된 타입이 섞임으로써 발생할 수 있는 런타임 형변환 예외를 사전 방어
- **선언 예시:** `ArrayList<String> list = new ArrayList<>();` (String 데이터만 보관 가능)

List 계열과 ArrayList

- **ArrayList**: 내부적으로 일반 배열을 품고 있지만, 데이터 추가/삭제 발생 시 스스로 빈방 크기를 알아서 확장하고 줄이는 고성능 동적 배열
- **순서와 중복**: 입력한 데이터의 인덱스 순서가 영구 유지되며, 데이터의 중복 저장을 아무런 제약 없이 허용
- **조회 효율**: 내부적으로 배열 기반이므로 인덱스를 통한 임의 요소 탐색 속도가 우수함

ArrayList 추가, 제거 및 크기 조회

```
1 import java.util.ArrayList;
2 public static void main(String[] args) {
3     ArrayList<String> list = new ArrayList<>();
4     list.add("Java");
5     list.add("Spring");
6     list.remove(0);
7     System.out.println(list.size()); // 1
8 }
```


Map 계열과 HashMap

- **HashMap**: 키(Key)와 값(Value)이 매칭되는 쌍으로 요소를 묶어 힙 영역에 관리하는 구조 (JS의 객체 리터럴과 유사)
- **키의 고유성**: 식별자 역할을 수행하는 키(Key)는 절대 중복될 수 없으며, 대입 대상인 값(Value)은 중복 저장이 가능
- **순서 비보장**: 저장소 내부에 요소들이 해시 함수를 통해 배치되므로, 데이터를 입력한 순서가 그대로 지켜지지 않음

HashMap 값 매핑 및 특정 키 조회

```
1 import java.util.HashMap;
2 public static void main(String[] args) {
3     HashMap<String, Integer> map = new HashMap<>();
4     map.put("Java", 100);
5     map.put("Spring", 90);
6     System.out.println(map.get("Java")); // 100
7     System.out.println(map.size()); // 2
8 }
```

Set 계열과 HashSet

- **HashSet**: 데이터의 저장 순서를 보장하지 않으며, 중복 데이터의 저장을 기계적으로 완전히 차단하는 독자적 집합 자료구조
- **중복 검출**: 데이터 삽입 시 내부적으로 `hashCode()` 와 `equals()` 연산을 수행하여 이미 같은 값이 있다면 삽입을 거절
- **고속 판별**: 컬렉션 내에 특정 원소가 들어있는지 포함 관계를 판별하는 탐색 연산 속도가 극도로 빠름

HashSet을 이용한 중복 없는 요소 제어

```
1 import java.util.HashSet;
2 public static void main(String[] args) {
3     HashSet<String> set = new HashSet<>();
4     set.add("Java");
5     set.add("Java"); // 중복 삽입 (무시됨)
6     System.out.println(set.contains("Java")); // true
7     System.out.println(set.size()); // 1
8 }
```